

P0906R0: Improvement suggestions for the Modules TS

Introduction

During CWG review of the Modules TS, I stumbled upon some corner cases that, in my view, result in surprising and inconsistent outcomes. This paper suggests changes to the Modules TS to reduce the surprises. In the examples below, I show the normative outcome according to the current Modules TS.

1 Different access paths to a type

For an exported entity, only the properties introduced or implicitly redeclared in exported declarations are reachable from the outside.

In contrast, properties of a non-exported entity are reachable from the outside according to the "attendant entities" rules in 10.7.6. A special error condition in paragraph 3 ensures a consistent view from the outside. Example:

```
export module M;
struct S;
export S f();      // #1; cannot call this function (incomplete return type)
struct S { };
export S g();      // error: attendant class type S has different properties vs. #1 (see 10.7.6p3)
```

From the outside (i.e. from a module or main program importing M), `decltype(f())` would access an incomplete S and `decltype(g())` would access the complete S, which would be inconsistent. The resolution in the Modules TS is to make a module giving rise to this situation ill-formed.

However, there are other situations where different access paths are resolved by essentially simply exporting the complete type. Example:

```
export module M;
struct C { };
struct S {
    struct B { };
    using C = ::C;
};
export S f();      // #1; S::B and ::C are not attendant entities
export C g();      // #2; ::C is an attendant entity

// elsewhere
import M;
int x = sizeof(decltype(f())::B); // error: incomplete class type B
int y = sizeof(decltype(f())::C); // ok: C is complete
```

By design, nested types are not attendant entities and thus the completeness of `S::B` or `S::C` is not reachable from the outside given only #1. However, the completeness can be added to the reachable properties by a later, unrelated declaration such as #2. In this case, the access path through #1 will see the complete type `S::C`.

It seems strange to treat the two examples differently. For the first example, instead of making the program ill-formed, an alternative resolution that preserves the consistency design goal is to simply consider S a complete type from the outside. The uniform rule would be "the properties of an entity reachable from the outside are the union of those made available through contributing declarations". This rule also addresses exported entities, where (of course) only *export-declarations* contribute.

It is recommended to treat this as a Defect Report against the Modules TS.

2 Definitions of inline functions

Module interface units serve two purposes: One, they offer a set of declarations visible to all module implementation units. These are all the declarations in the purview of the module interface unit. Two, they offer a set of declarations (entities, properties) visible to the outside. These are the exported declarations plus their attendant entities. For a given entity such as a class type or a function, it is possible to export a subset of the properties by forward-declaring with `export` and later redeclaring (or defining) without `export`:

```
export module M;

export int f(int);
int f(int = 5);

export struct S;
struct S { int x; };

export constexpr int g(int);
constexpr int g(int x) { return x+1; }

// elsewhere
import M;
int x = f();           // error: default argument not visible
S y;                  // error: S is not complete
constexpr int z = g(42); // ok
```

This allows programmer control of the properties visible from the outside.

However, this does not apply to definitions of inline functions. If an inline function is exported, its definition must always appear in the module interface unit (10.1.2 [dcl.fct.spec]) and the definition itself is implicitly exported (10.7.6p1 [dcl.module.reach]). (It is understood that a definition of an inline function must be reachable whenever it is called.)

```
export module M;
export class S {
public:
    // public, exported interface
protected:
    void f();    // helper function, exported "by accident"
};
export class D : public S {
public:
    void g();
};
```

Assume that `S::f` should get an inline definition, but will only be called from a single module implementation unit (the one implementing `D::g`). It seems the only currently valid place where to put the inline definition is the module interface unit, also implicitly making that inline definition exported. However, to implement the use-case, it would be sufficient (and expose fewer things) to put the inline definition into the module implementation unit for `D::g`.

Further, a module importing `M` could inspect `S::f` for its type (return type, parameter types), and such inspection would not need the definition of `S::f`.

The reasoning that supports selectively exporting default function arguments or class type definitions should also apply here: Give the programmer control of what is and is not exported.

Since `constexpr` functions are implicitly inline, a change here does also apply to `constexpr` functions.

It is recommended to treat this as a Defect Report against the Modules TS.

3 Allow vacuous exports

The keyword `export` can be used to export a single declaration, or a brace-enclosed sequence of declarations. However, the syntactic treatment is different from linkage specifications [dcl.link], a similar context where such a construction is allowed.

```

extern "C++" {
    int f();
    static_assert(1 == 1, "blah");    // ok
    static int g();                  // ok; linkage specification does not apply
}

export {
    int f();
    static_assert(1 == 1, "blah");    // error: does not declare a name
    static int g();                  // error: cannot export function declared static
}

```

A linkage specification is ignored for all declarations to which it does not apply. In contrast, attempting to export a declaration that cannot be exported (e.g. a `static_assert`) makes the program ill-formed.

Assuming that both constructs are intended to change a given property of a possibly large set of pre-existing declarations with minimal syntax, I suggest to allow such vacuous exports. Pre-existing sets of declarations that are converted to modules might reasonably contain `static_asserts` or unnamed namespaces in the middle of to-be-exported declarations. Rearranging the code might be too much of a burden.

It is recommended to treat this as a Defect Report against the Modules TS.

4 Exclude private members from attendant entities

The current rules in 10.7.6 [dcl.module.reach] paragraph 2 do not differentiate between public, protected, and private members when defining the set of attendant entities, although properties of private members cannot be discovered from outside the defining module. Example:

```

export module M;
struct S { };
class C {
public:
    void f();
private:
    S g();    // helper function
};
export C h();    // S is among the attendant entities

```

There is no way to name `C::g()` from outside the module defining `C`, thus there is no way to name the return type of `C::g()`. Consistent with the approach to clearly delineate what is visible from the outside, I suggest to remove consideration of private members for attendant entities.

It is recommended to treat this as a Defect Report against the Modules TS.

Wording changes for #1, #2, and #4

Change in 10.7.6 [dcl.module.reach] paragraph 3:

If `X` is an attendant entity of two exported declarations designating two distinct entities, its reachable semantic properties ~~shall be the same at the points where the declarations occur~~ **are the union of those introduced where the declarations occur**. [Example:

```

export module M;
struct S;
export S f();    // #1
struct S { };
export S g();    // error: class type S has different properties from #1 ok

// translation unit 2
import M;
decltype(f()) x;    // ok, entity S is a complete type

```

-- end example]

Editorial note: Maybe this rule can be folded into paragraph 2.

Drop the change to 10.1.2 [dcl.fct.spec]:

~~An exported inline function shall be defined in the same translation unit containing its export declaration. [Note: There is no restriction on the linkage (or absence thereof) of entities that the function body of an exported inline function can reference. A constexpr function (10.1.5) is implicitly inline. --- end note]~~

Remove in 10.7.6 [dcl.module.reach] paragraph 1 bullet 1:

- ... ~~Furthermore, if D denotes an inline function, the property that the inline function has a definition (10.1.2) is a reachable semantic property, even if that definition is not exported.~~ Otherwise,

Change in 10.7.6 [dcl.module.reach] paragraph 2, second list, bullet 3:

- ...
- If T is a class type owned by M, the set of attendant entities includes T itself, the union of the sets of the attendant entities determined by its direct **public or protected** base classes owned by M, the sets of the attendant entities of its data members, static data member templates, member functions, member function templates, the function parameters of its constructors and constructor templates, **as far as these members are public or protected**. Furthermore, if T is a class template specialization, the set of attendant entities also includes: the class template if it is owned by M, the union of the sets of attendant entities determined by the type template-arguments, the sets of the attendant entities of the templates used as template template-arguments, the sets of the attendant entities determined by the types of the non-type template-arguments.
- ...